DIGT 3203

Distributed Systems

Unit 1

Introduction to Distributed Systems & System Models

UNIT 1 - PART 1

Introduction to Distributed Systems

Making code do what it's supposed to do is the easy part. Handling the myriad ways it can go wrong is the real work.

- @ajlozier

Welcome

Welcome to the

Distributed Systems course

I wholeheartedly wish you a productive semester!!

General Structure

- Lectures
  - Block week: in person and labs
  - Non-block week: on-line and home assignments / mini projects
- Marking
  - Marks are built cumulatively per unit based on quizzes and assignments / mini projects

Topics

- CHARACTERIZATION OF DISTRIBUTED SYSTEMS
- SYSTEM MODELS
- NETWORKING AND INTERNETWORKING
- INTERPROCESS COMMUNICATION
- REMOTE INVOCATION / SERVICE COMPUTING
- INDIRECT COMMUNICATION
- OPERATING SYSTEM SUPPORT
- PEER-TO-PEER SYSTEMS
- SECURITY
- DISTRIBUTED FILE SYSTEMS
- NAME SERVICES
- TIME AND GLOBAL STATES
- COORDINATION AND AGREEMENT
- TRANSACTIONS AND CONCURRENCY CONTROL
- DISTRIBUTED TRANSACTIONS
- REPLICATION

Background
Hands-on
Core Topics

Distributed Systems Definition

A distributed system is one in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages.

These systems

Exhibit a high degree of *Concurrency*

There is no *Global clock*

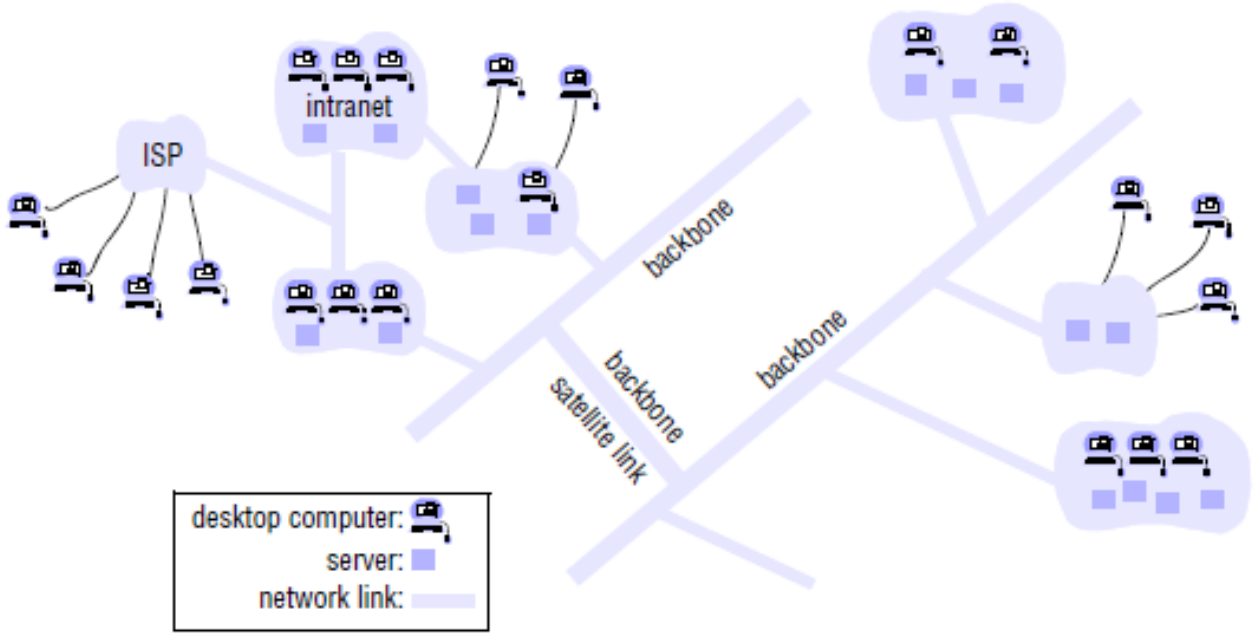
Are subject to *Independent failures*

Distributed Systems Another Definition

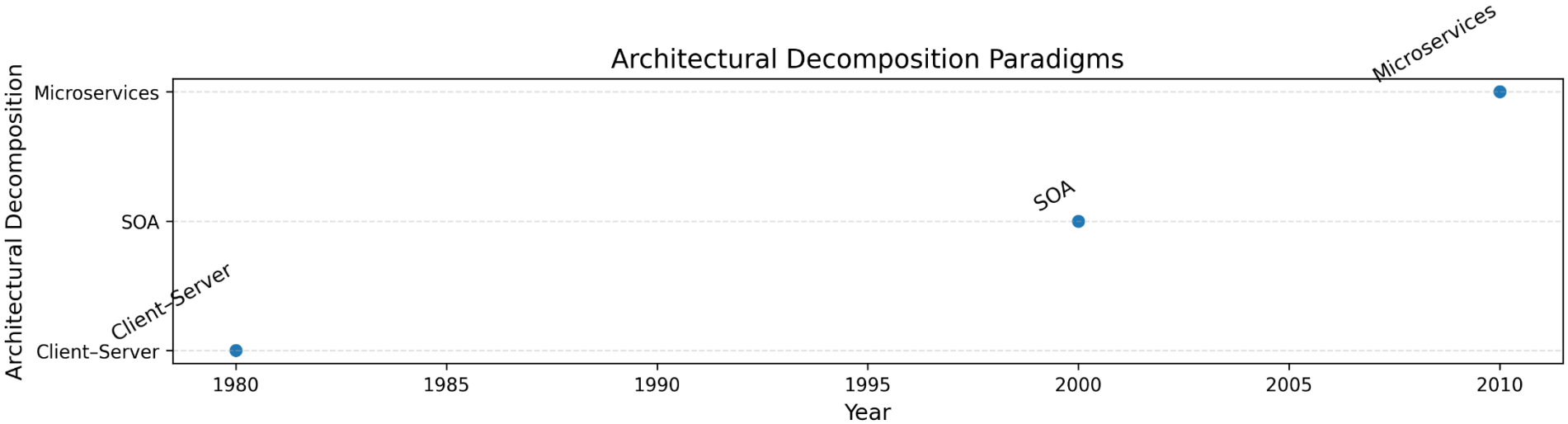
*A distributed system is a collection of independent computers (**nodes**) that work **together** over a network, appearing to users **as a single, cohesive system** to achieve a common **goal**, offering benefits like scalability, reliability, and fault tolerance by sharing tasks and resources, fundamental to the internet and modern cloud applications.*

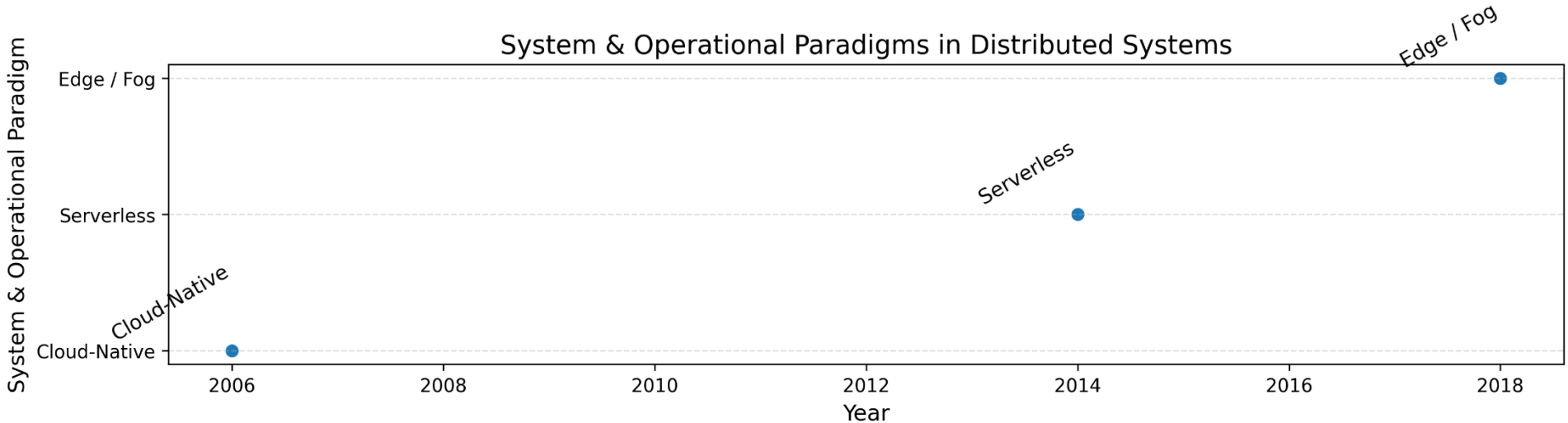
Key characteristics include multiple autonomous components, network communication, coordination, scalability (adding nodes), and fault tolerance (handling individual node failures).

A Typical Internet-work Structure



Timeline of Paradigms





Architectural Paradigms Comparison

Architectural Decomposition Paradigms

Paradigm	Unit of Design	Coupling	Deployment Unit	Failure View
Client–Server	Application	Tight	Application	Crash
SOA	Service	Moderate	Service	Partial failure
Microservices	Microservice	Loose	Microservice	Failure expected

System & Operational Paradigms

Paradigm	Primary Concern	Infrastructure Assumption	Failure Model	Responsibility
Cloud-Native	Elasticity & resilience	Ephemeral	Failure assumed	Platform
Serverless	Zero ops	Fully managed	Continuous failure	Platform
Edge / Fog	Latency & locality	Highly distributed	Intermittent failure	Hybrid

Microservices vs. Serverless

- Microservices are an architectural style in which an application is decomposed into independent services, each responsible for a specific business capability.

- Serverless is an execution and deployment model where developers run short-lived functions without managing servers or runtime infrastructure.

- In modern systems, it is common to see Microservices at the architectural level, implemented using a mix of containers and serverless functions.

- Serverless is an execution and deployment model where developers run short-lived functions without managing servers or runtime infrastructure.
  - Code is executed as stateless functions (Function-as-a-Service).
  - Functions are event-triggered and ephemeral.
  - Scaling, fault tolerance, and infrastructure management are fully handled by the cloud provider.
  - Billing is based on execution time, not uptime.
  - State is externalized to managed services.

Some Key Issues

- Concurrency
- No global clock
- Independent failures

Service

- We use the term ***service*** for a distinct part of a computer system that manages a collection of related resources and presents their functionality to users and applications.

- Resources in a distributed system are physically encapsulated within computers and can only be accessed from other computers by means of communication. For effective sharing, each resource must be managed by a program that offers a communication interface enabling the resource to be accessed and updated reliably and consistently.

Servers and Clients

- The term ***server*** refers to a **running program (a process)** on a networked computer that accepts requests from programs running on other computers to perform a service and responds appropriately.

- The requesting processes are referred to as ***clients***, and the overall approach is known as client-server computing.

- Requests are sent in messages from clients to a server and replies are sent in messages from the server to the clients.

- A **complete interaction** between a client and a server, from the point when the client sends its request to when it receives the server's response, is called a ***remote invocation***.

- The same process may be both a client and a server. The terms 'client' and 'server' apply only to the roles played in a single request.

Concurrency vs. Parallelism

- ***Concurrency*** is about dealing with multiple tasks at once by interleaving their execution; tasks make progress independently but not necessarily at the same time.

- ***Parallelism*** is about doing multiple tasks at the same time, typically by executing them simultaneously on multiple cores, CPUs, or machines.

- A system can be concurrent without being parallel (e.g., a single-core CPU switching between tasks), but parallel systems are always concurrent.

- Concurrency focuses on *correctness*, *coordination*, and *synchronization*, while parallelism focuses on *performance* and *speed-up*.

- In distributed and cloud systems, concurrency is essential for scalability and responsiveness, whereas parallelism is key for throughput and computational efficiency.

UNIT 1 - PART 2

Distributed Systems Key Challenges

Making code do what it's supposed to do is the easy part. Handling the myriad ways it can go wrong is the real work.

- @ajlozier

Challenges

- *Heterogeneity*: Distributed systems must be constructed from a variety of different networks, operating systems, computer hardware and programming languages and implementations by different developers.

- *Openness*: Distributed systems should be extensible and expose services through APIs and middleware frameworks.

- *Security*: Encryption can be used to provide adequate protection of shared resources and to keep sensitive information secret when it is transmitted in messages. Compliance with policies is another encompassing issue.

- *Scalability*: A distributed system is scalable if adding users requires only constant additional resources, enabled by bottleneck-free algorithms, hierarchical data organization, and data replication.

Challenges (Continued)

- *Failure handling*: each component needs to be aware of the possible ways in which the components it depends on may fail and be designed to deal with each of those failures appropriately.

- *Concurrency*: Each resource must be designed to be safe in a concurrent environment.

- *Transparency*: Make certain aspects of distribution invisible to the application programmer so that they need only be concerned with the design of their particular application.

- *Quality of service*: Provide guarantees regarding the qualities associated with such service access. Examples of such qualities include parameters related to performance, security and reliability.

Heterogeneity

- The Internet connects heterogeneous networks whose differences are hidden by common Internet protocols, though each network requires its own protocol implementation.

- Communication across systems is complicated by differences in hardware representations, operating-system APIs, and programming-language data models.

- Middleware builds on Internet protocols to further mask OS, hardware, and language heterogeneity, providing higher-level programming abstractions. It offers uniform computational models (e.g., remote invocation, messaging, transactions) that hide low-level message passing.

- Interoperable communication between independently developed programs is possible only through agreed-upon standards for protocols and data representation.

- Mobile code and virtual machines (e.g., JVM) improve portability by enabling code execution across heterogeneous platforms.

Openess

- Openness is the ability of a distributed system to be extended and reimplemented, primarily by adding new resource-sharing services usable by many clients.
- Openness depends on published and well-documented interfaces, enabling independent developers to build, extend, and integrate components.
- Publishing interfaces (often outside formal standardization) must be combined with careful design to manage the complexity of heterogeneous, multi-developer systems.
- The Internet exemplifies openness through RFCs and web standards (e.g., W3C), which enabled interoperable protocols and applications such as email and the Web.
- Open distributed systems support hardware and software extensibility, promote vendor independence, and require strict conformance to common standards to ensure correct operation.

Security

- Security is critical in distributed systems because many shared information resources have high value; it is defined by **confidentiality**, **integrity**, and **availability**.

- Although the Internet enables global communication, unrestricted access introduces security risks that firewalls alone cannot address, especially for internal misuse and external services.

- Secure communication requires both **protecting message contents** and **verifying identities of users and services** (e.g., doctors accessing records, customers interacting with banks).

- Encryption techniques are widely used to ensure confidentiality and authentication, but they do not solve all security problems.

- Two major unresolved challenges remain: *denial-of-service attacks*, which disrupt service availability, and *mobile code security*, where executing downloaded code can lead to unpredictable and harmful behavior.

Scalability

- Scalability is the ability of a distributed system to remain effective as the number of users and resources grows, from small intranets to the global Internet.

- Scalable systems must control resource costs, ensuring that required physical resources grow proportionally with demand (i.e., at most $O(n)$O(n) for nn users).

- They must also limit performance degradation, favoring decentralized and hierarchical algorithms (e.g., $O(\log n)$ access time) over centralized or linear ones.

- Scalability requires avoiding resource exhaustion and bottlenecks, as illustrated by IP address depletion and early centralized naming systems that did not scale.

- Proven scalability techniques include *decentralization*, *replication*, *caching*, and the use of *multiple servers*, reducing bottlenecks and enabling concurrent processing as systems grow.

Failure Handling

- Failures are inevitable in computer systems; in distributed systems they are typically partial, meaning some components fail while others continue to operate, making failure handling particularly challenging.

- Failure detection is possible in some cases (e.g., checksums for corrupted data), but many failures—such as remote server crashes—are difficult or impossible to detect reliably.

- Failure masking reduces the impact of detected failures through techniques like message retransmission and data replication, though these methods are not guaranteed to succeed in all cases.

- **Failure tolerance and recovery** are essential: systems and clients are designed to continue operating despite failures, inform users appropriately, and recover consistent system state after crashes (e.g., rollback mechanisms).

- *Redundancy* and *replication* are key strategies for achieving high availability, ensuring continued service despite component failures, although keeping replicas consistent without performance loss remains a major challenge.

Concurrency

- Services and applications share resources in distributed systems, creating the possibility of simultaneous access by multiple clients.
- Concurrency ≠ Parallelism
- To improve throughput, systems typically process client requests concurrently rather than one at a time.
- Concurrent execution can lead to conflicting operations and inconsistent results if access to shared resources is uncontrolled.
- Any object representing a shared resource must therefore be designed to operate correctly in a concurrent environment, whether in servers or applications.
- Synchronization mechanisms (e.g., semaphores) are required to maintain data consistency when operations execute concurrently.

Transparency

- Transparency conceals the distribution of components in a distributed system, allowing users and programmers to **perceive** it as a single, unified system; it strongly influences system design.

- Reference models such as ANSA and RM-ODP identify multiple forms of transparency, with access and location transparency being the most important (often grouped as network transparency).

- Access transparency allows uniform operations on local and remote resources (e.g., file APIs, GUIs), while location transparency hides the physical or network location of resources (e.g., email addresses, URLs).

- Some identifiers (e.g., URLs with domain names) limit mobility and replication transparency, as they bind resources to specific locations or hosts.

- Additional transparencies include failure transparency (masking faults via retries and exceptions) and mobility transparency (hiding movement of clients or resources, as in mobile phones).

Types of Transparency

- **Access transparency** allows local and remote resources to be accessed in the same way.
- **Location transparency** enables resource access without knowing their physical or network location.
- **Concurrency transparency** allows multiple processes to share resources safely and simultaneously.
- **Replication transparency** hides the existence of multiple resource copies used for reliability and performance.
- **Failure transparency** masks hardware or software failures so tasks can still complete.
- **Mobility transparency** allows resources and clients to move without affecting system operation.
- **Performance transparency** enables dynamic reconfiguration to maintain performance under varying loads.
- **Scaling transparency** allows systems and applications to grow without changing their structure or algorithms.

Quality of Service

- QoS focuses on ensuring that computing and network resources are available when needed to meet deadlines.
- Quality of Service (QoS) concerns the nonfunctional properties of a system, including **reliability**, **security**, **performance**, and **adaptability**.
- Performance is no longer defined only by responsiveness and throughput, but by the ability to meet timeliness guarantees.
- Time-critical applications (e.g., multimedia streaming) require data to be processed and delivered at fixed rates to meet user expectations.
- Achieving QoS requires resource reservation and management, with guarantees provided only when sufficient resources are available.

UNIT 1 - PART 3

Process Communication

Making code do what it's supposed to do is the easy part. Handling the myriad ways it can go wrong is the real work.

- @ajlozier

Applications Communication

- Applications expose an API (i.e. a collection of related operations – e.g. cart management operations in a e-commerce scenario), or end-points that receive messages (read/write).
- Applications communicate by **exchanging messages** (request messages, reply messages, control/notification messages) via their APIs.
- The simplest form of communication is one application to now the exact address and API of the other (too tight coupling).
- Alternative forms require a framework that sits in the middle (hence middleware) that facilitates communication and relieves the two communicating parties to know where each other and what their API is.

Communication Protocols

- A **communication protocol** in a distributed system is a **formal specification of rules** that defines **how independent components exchange messages**, including:
  - Message **format** (syntax)
  - Message **meaning** (semantics)
  - Message **ordering and timing**
  - **Error handling** and recovery behavior

- In short: a communication protocol ensures that distributed components can **understand each other and coordinate correctly**, despite failures, delays, and concurrency.

- **Protocol**: Defines what messages are exchanged and in what order.

- **Algorithm**: Defines the logic performed upon receiving messages.

Core Elements of a Communication Protocol

- **Message Structure**
  Defines how messages are encoded (fields, headers, payload).

- **Interaction Pattern**
  Defines who talks to whom and when (request–response, publish–subscribe, streaming).

- **State Machine**
  Defines valid message sequences and transitions.

- **Failure Semantics**
  Defines how to handle loss, duplication, reordering, and timeouts.

Messages

- A **message** in a distributed system is a **self-contained unit of data** sent from one process to another to **communicate information, request an action, or coordinate behavior**, typically over an unreliable network.

A message encapsulates:

- **Data** being communicated
- **Control information** (metadata)
- **Context** needed for correct interpretation

- In distributed systems, **all interaction happens via message exchange—there is no shared memory.**

Typical Structure of a Message

Component	Description
Header	Source, destination, message type, protocol version
Payload	Application data
Metadata	Timestamp, message ID, correlation ID
Optional	Security info (signature, token)

Core Characteristics of a Message

- **Self-contained**
  A message carries all information needed to be understood by the receiver.

- **Asynchronous by nature**
  Sending and receiving may occur at different times.

- **Subject to failures**
  Messages may be delayed, lost, duplicated, or reordered.

- **Typed and structured**
  Defined by a schema or protocol.

Examples of Messages

HTTP Call

http://orders.service.com/orders/123

Payload (Command Message)

```
{
  "command": "ShipOrder",
  "orderId": "123"
}
```

Payload (Event Message)

```
{
  "eventType": "OrderCreated",
  "orderId": "123",
  "timestamp": "2025-03-12T10:15:00Z"
}
```

Payload (Heartbeat Message)

```
{
  "type": "HEARTBEAT",
  "nodeId": "node-7"
}
```

Application Integration

- **Application integration** in distributed systems is the practice of **enabling independent, heterogeneous applications to communicate, coordinate, and share data** in order to support **end-to-end business or system functionality**, while preserving their **autonomy and internal implementation details**.

- In essence, application integration allows *separate systems to work together as one logical system*.

Common Application Integration Styles

***Point-to-Point Integration*:** Direct connections between applications.
Example: A payroll system directly calls an HR system via an API.
Pros: Simple to implement.
Cons: Does not scale; creates tight coupling.

Service-Oriented Integration (SOA): Applications expose functionality as services. Integration via a central mediation layer
Example: An order service, billing service, and shipping service integrated through an ESB using SOAP.
Key Technologies: SOAP, WSDL, ESB

API-Based Integration: Applications integrate through well-defined REST or GraphQL APIs.
Example: A mobile app integrates with backend services via REST APIs.
Key Technologies: HTTP/REST, JSON, OAuth

Message-Based (Event-Driven) Integration: Applications exchange messages or events asynchronously.
Example: An e-commerce system publishes an *OrderPlaced* event; inventory and billing systems subscribe.
Key Technologies: Kafka, RabbitMQ, AMQP

Data-Level Integration: Applications share data through databases or data pipelines.
Example: Multiple systems reading from a shared data warehouse.
Risk; Tight coupling and hidden dependencies.

Workflow / Process Integration: Integration at the business-process level.
Example: A BPM engine orchestrates order fulfillment across CRM, ERP, and logistics systems.
Key Technologies: BPMN, workflow engines

Middleware

- **Middleware** is a **software layer that sits between applications and the underlying operating system, network, and hardware**, providing **common services and abstractions** that simplify the development, integration, and operation of distributed applications.

- Middleware hides distribution complexity and provides programmers with a **higher-level programming model** for building distributed systems.

- Middleware addresses Network communication, Partial failures, Concurrency, and Heterogeneity (languages, platforms, OSs) challenges by providing:

  - **Location transparency**
  - **Communication abstraction**
  - **Fault handling**
  - **Scalability support**
  - **Interoperability**

Middleware Responsibilities

- **Communication abstraction:** Hides low-level message passing (sockets).
- **Naming and discovery:** Locates services dynamically.
- **Serialization and data transformation:** Converts in-memory data to wire formats.
- **Reliability and fault tolerance:** Retries, timeouts, failover.
- **Security:** Authentication, authorization, encryption.
- **Transaction and coordination support:** Ensures consistency across components.

Concepts at a Glance

Protocol vs Architecture vs Middleware

Concept	Defines	Concern	Examples
Communication Protocol	Rules of message exchange	Syntax, order, failure handling	TCP, HTTP, Raft
Architecture	System structure & components	Decomposition & interaction	Client–Server, Microservices
Middleware	Abstractions over communication	Transparency, reuse	RPC, Message Brokers

Distributed Computing as a Utility

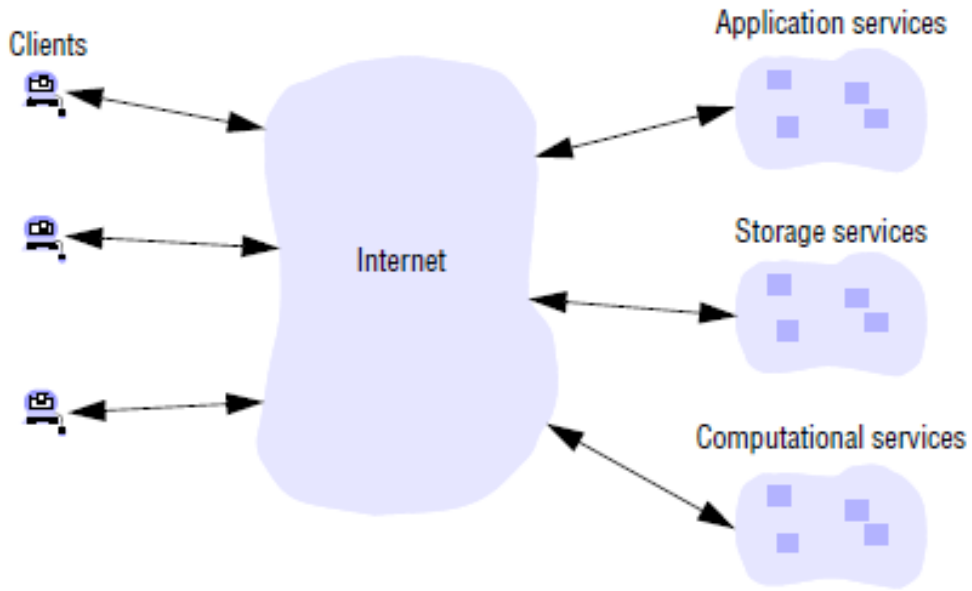
- Resources are provided by appropriate service suppliers and effectively rented rather than owned by the end user. This model applies to both physical resources and more logical services.

- A typical example is *cloud computing* which captures this vision of computing as a utility.

- A cloud is defined as a set of Internet-based application, storage and computing services sufficient to support most users' needs, thus enabling them to largely or totally dispense with local data storage and application software.

- The cloud computing also promotes a view of everything as a service, from physical or virtual infrastructure through to software, often paid for on a per-usage basis rather than purchased.

Cloud Computing Schematic



The cloud computing also promotes a view of everything as a service, from physical or virtual infrastructure through to software, often paid for on a per-usage basis rather than purchased.

Summary

- Distributed systems are ubiquitous, ranging from the global Internet and organizational intranets to mobile and wireless computing environments.
- Their primary motivation is resource sharing, where servers manage shared resources and clients access them over a network.
- Distributed systems must handle significant heterogeneity, masking differences in hardware, operating systems, networks, and programming languages.
- Key design challenges include openness, security, scalability, failure handling, and concurrency, each requiring careful architectural support.
- Transparency aims to hide distribution details from application developers while still requiring proper handling of failures and exceptions.
- Beyond functionality, distributed systems must provide quality-of-service guarantees, including performance, reliability, and security assurances.